

Reverse_Hard_01 (планировался как сложный)

Название: Это был я, Дио!! / Kono Dio Da!!

Описание: Мой последний изумрудный всплеск... Моё сообщение Вам... Большого я не смогу, Джостар-сан. Поймите его, передайте другим...

Флаг: FECTF{Z4_W4rud0}

Решение:

Название и описание задания уже сами по себе содержат подсказку – они отсылают к персонажу Дио из Приключений ДжоДжо, способностью которого была остановка времени. Значит, что-то связано со временем

Открываем программу в гидре, смотрим определенные строки. Находим следующее

140024870	Enter password:	"Enter password: "	ds
140024888	Congratulations! FECTF{	"Congratulations! FECTF{"	ds
1400248a8	Wrong password! Try again.	"Wrong password! Try again."	ds

Переходим по адресу использования, видим следующее:

```

puVar6 = local_lb8;
for (lVar5 = 0x3a; lVar5 != 0; lVar5 = lVar5 + -1) {
    *puVar6 = 0xffffffff;
    puVar6 = puVar6 + 1;
}
local_20 = DAT_140029040 ^ (ulonglong)local_lb8;
__CheckForDebuggerJustMyCode(&DAT_14002f109);
bVar1 = thunk_FUN_140018580();
if (bVar1) {
    pbVar3 = (basic_ostream<> *)
        thunk_FUN_140013a10((longlong *)cout_exref, "Debugger detected! Exiting...");
    std::basic_ostream<>::operator<<
        (pbVar3, (_func_basic_ostream<>_ptr_basic_ostream<>_ptr *)&LAB_140011050);
}
else {
    bVar1 = thunk_FUN_140017fb0();
    if (bVar1) {
        thunk_FUN_140013a10((longlong *)cout_exref, "Enter password: ");
        thunk_FUN_140015880(local_lb0);
        thunk_FUN_140013550((longlong *)cin_exref, local_lb0);
        thunk_FUN_140018410(local_l70);
        cVar2 = thunk_FUN_140013eb0((longlong)local_lb0, (longlong)local_l70);
        if (cVar2 == '\0') {
            pbVar3 = (basic_ostream<> *)
                thunk_FUN_140013a10((longlong *)cout_exref, "Wrong password! Try again.");
            std::basic_ostream<>::operator<<
                (pbVar3, (_func_basic_ostream<>_ptr_basic_ostream<>_ptr *)&LAB_140011050);
        }
        else {
            thunk_FUN_1400188c0(local_l30);
            plVar4 = (longlong *)thunk_FUN_140013a10((longlong *)cout_exref, "Congratulations! FECTF{");
            plVar4 = (longlong *)thunk_FUN_140013990(plVar4, (longlong)local_l30);
            pbVar3 = (basic_ostream<> *)thunk_FUN_140013a10(plVar4, "}");
            std::basic_ostream<>::operator<<
                (pbVar3, (_func_basic_ostream<>_ptr_basic_ostream<>_ptr *)&LAB_140011050);
            thunk_FUN_1400161f0(local_l30);
        }
        system("pause");
        local_34 = 0;
        thunk_FUN_1400161f0(local_l70);
        thunk_FUN_1400161f0(local_lb0);
    }
    else {
        pbVar3 = (basic_ostream<> *)
            thunk_FUN_140013a10((longlong *)cout_exref, "Code integrity check failed!");
        std::basic_ostream<>::operator<<
            (pbVar3, (_func_basic_ostream<>_ptr_basic_ostream<>_ptr *)&LAB_140011050);
    }
}
_RTC_CheckStackVars((longlong)local_ld8, (int *)&DAT_1400242a0);
thunk_FUN_140019a50(local_20 ^ (ulonglong)local_lb8);
return;
}

```

Есть проверка на дебаггер, есть проверка на целостность (по крайней мере, так задумывалось)

Рассмотрим механизм запроса пароля и его валидации

```

bVar1 = thunk_FUN_140017fb0();
if (bVar1) {
    thunk_FUN_140013a10((longlong *)cout_exref, "Enter password: ");
    thunk_FUN_140015880(local_1b0);
    thunk_FUN_140013550((longlong *)cin_exref, local_1b0);
    thunk_FUN_140018410(local_170);
    cVar2 = thunk_FUN_140013eb0((longlong)local_1b0, (longlong)local_170);
    if (cVar2 == '\0') {
        pbVar3 = (basic_ostream<> *)
            thunk_FUN_140013a10((longlong *)cout_exref, "Wrong password! Try again.");
        std::basic_ostream<>::operator<<
            (pbVar3, (_func_basic_ostream<>_ptr_basic_ostream<>_ptr *)&LAB_140011050);
    }
    else {
        thunk_FUN_1400188c0(local_130);
        plVar4 = (longlong *)thunk_FUN_140013a10((longlong *)cout_exref, "Congratulations! FECTF{");
        plVar4 = (longlong *)thunk_FUN_140013990(plVar4, (longlong)local_130);
        pbVar3 = (basic_ostream<> *)thunk_FUN_140013a10(plVar4, "");
        std::basic_ostream<>::operator<<
            (pbVar3, (_func_basic_ostream<>_ptr_basic_ostream<>_ptr *)&LAB_140011050);
        thunk_FUN_1400161f0(local_130);
    }
}

```

Посмотрим что в функции, значение которой в bVar1

```

bool FUN_140017fb0(void)
{
    byte local_114;
    int local_f4;

    __CheckForDebuggerJustMyCode(&DAT_14002f109);
    local_114 = 0;
    for (local_f4 = 0; local_f4 < 0x20; local_f4 = local_f4 + 1) {
        local_114 = local_114 ^ (byte)thunk_FUN_140018410[local_f4];
    }
    return local_114 != 0;
}

```

Видим ещё одну функцию, посмотрим что там происходит

```

2 void thunk_FUN_140018410(longlong *param_1)
3
4 {
5     int iVar1;
6     longlong lVar2;
7     undefined4 *puVar3;
8     undefined1 auStack_1e8 [32];
9     undefined4 auStack_1c8 [2];
10    __time64_t a_stack_1c0 [4];
11    longlong lStack_1a0;
12    uint uStack_184;
13    int iStack_124;
14    uint uStack_44;
15    undefined8 *puStack_30;
16    ulonglong uStack_28;
17
18    puVar3 = auStack_1c8;
19    for (lVar2 = 0x3e; lVar2 != 0; lVar2 = lVar2 + -1) {
20        *puVar3 = 0xc0000000;
21        puVar3 = puVar3 + 1;
22    }
23    uStack_28 = DAT_140029040 ^ (ulonglong)auStack_1c8;
24    uStack_44 = 0;
25    __CheckForDebuggerJustMyCode(&DAT_14002f109);
26    a_stack_1c0[0] = FUN_140018fd0((__time64_t *)0x0);
27    lStack_1a0 = FUN_140018d30(a_stack_1c0);
28    uStack_184 = *(uint *) (lStack_1a0 + 8);
29    srand(uStack_184);
30    puStack_30 = thunk_FUN_140015880(param_1);
31    uStack_44 = uStack_44 | 1;
32    for (iStack_124 = 0; iStack_124 < 8; iStack_124 = iStack_124 + 1) {
33        iVar1 = rand();
34        thunk_FUN_140016b50(param_1, (char) ((longlong)iVar1 % 0x1a) + 'A');
35    }
36    _RTC_CheckStackVars((longlong)auStack_1e8, (int *)&DAT_140024170);
37    thunk_FUN_140019a50(uStack_28 ^ (ulonglong)auStack_1c8);
38    return;
39 }

```

Выходит, пароль состоит из 8 символов, которые генерируются случайно по следующей формуле

```

thunk_FUN_140016b50(param_1, (char) ((longlong)iVar1 % 26) + 'A');

```

Остается вопрос – а какой сид? А сид вот

```

local_1c0[0] = FUN_140018fd0((__time64_t *)0x0);
local_1a0 = FUN_140018d30(local_1c0);
local_184 = *(uint *) (local_1a0 + 8);
srand(local_184);

```

Значит, берется время. Вообще по сдвигу 8 в структуре `time_t` стоит час, значит сид – текущий час.

Воспроизводим функцию генерации

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <string>
#include <ctime>
#include <cstdlib>
#include <vector>
#include <algorithm>
using namespace std;

std::string generate_password() {
    time_t now = time(nullptr);
    tm* local_time = localtime(&now);
    int seed = local_time->tm_hour;
    srand(seed);
    std::string password;
    for (int i = 0; i < 8; ++i) {
        password += 'A' + (rand() % 26);
    }
    return password;
}

int main() {
    std::string pass = generate_password();
    std::cout << pass << std::endl;
}
```

Получим пароль, введём и выведется флаг

```
Enter password: DWUQQURL
Congratulations! FECTF{Z4_W4rud0}
Для продолжения нажмите любую клавишу . . . |
```

Рассмотрим другие методы решения

В этом участке кода проверяется отладка

```
bVar1 = thunk_FUN_140018580();
if (bVar1) {
    pbVar3 = (basic_ostream<> *)
        thunk_FUN_140013a10((longlong *)cout_exref, "Debugger detected! Exiting...");
    std::basic_ostream<>::operator<<
        (pbVar3, (_func_basic_ostream<>_ptr_basic_ostream<>_ptr *)&LAB_140011050);
}
```

Попробуем пропатчить на пропуск отладки

140018dc3 0f b6 c0	MOVZX	EAX,AL
140018dc6 85 c0	TEST	EAX,EAX
140018dc8 74 2e	JZ	LAB_140018df8
140018dca 48 8d 15	LEA	RDX,[s_Debugger_detected!_Exiting..._140024820= "Debu
4f ba 00 00		
140018dd1 48 8b 0d	MOV	RCX,qword ptr [->MSVCP140D.DLL::std::cout = 0002e
a0 43 01 00		
140018dd8 e8 eb 82	CALL	thunk_FUN_140013a10 undefin
ff ff		
140018ddd 48 8d 15	LEA	RDX,[LAB_140011050

Находим необходимый участок

00007FF746CA80B8	E8 5F89FFFF	call rev_hard_01 - копия.7FF746CA171C
00007FF746CA80B9	90	nop
00007FF746CA80BE	E8 5887FFFF	call rev_hard_01 - копия.7FF746CA151E
00007FF746CA80C3	0FB6C0	movzx eax,al
00007FF746CA80C6	85C0	test eax,eax
00007FF746CA80C8	74 2E	jz rev_hard_01 - копия.7FF746CA80F8
00007FF746CA80CA	48 8D15 4FBA0000	lea rdx,qword ptr ds:[<"Debugger detected! Exiting...">]
00007FF746CA80D1	48 8B0D A0430100	mov rcx,qword ptr ds:[<class std::basic_ostream<char, struct std::char_traits<char>> std::cout>]
00007FF746CA80D8	E8 EB82FFFF	call rev_hard_01 - копия.7FF746CA10C8
00007FF746CA80DD	48 8D15 6C82FFFF	lea rdx,qword ptr ds:[7FF746CA1050]
00007FF746CA80E4	48 8BC8	mov rcx,rcx

Нопаем оба вызова и меняем je на jne. Сохраняем патч. В этом же патче, или в следующем, если захочется поизучать программу в отладке, меняем валидацию проверки пароля

00007FF645518E6F	48 8D40 08	lea rcx,qword ptr ss:[rbp+8]
00007FF645518E73	E8 6883FFFF	call patched_3.7FF6455111E0
00007FF645518E78	0FB6C0	movzx eax,al
00007FF645518E7B	85C0	test eax,eax
00007FF645518E7D	74 5E	jne patched_3.7FF6455118DD
00007FF645518E7F	48 8D8D 88000000	lea rcx,qword ptr ss:[rbp+88]
00007FF645518E86	E8 3783FFFF	call patched_3.7FF6455111C2
00007FF645518E8B	90	nop
00007FF645518E8C	48 8D15 F5B90000	lea rdx,qword ptr ds:[<"Congratulations! FECTF{...">]
00007FF645518E93	48 8B0D DE420100	mov rcx,qword ptr ds:[<class std::basic_ostream<char, struct std::char_traits<char>> std::cout>]
00007FF645518E9A	E8 2982FFFF	call patched_3.7FF6455110C8
00007FF645518E9F	48 8D95 88000000	lea rdx,qword ptr ss:[rbp+88]
00007FF645518EA6	48 8BC8	mov rcx,rcx
00007FF645518EA9	E8 C987FFFF	call patched_3.7FF645511677
00007FF645518EAE	48 8D15 13BA0000	lea rdx,qword ptr ds:[<"...">]
00007FF645518EB5	48 8BC8	mov rcx,rcx
00007FF645518EB8	E8 0882FFFF	call patched_3.7FF6455110C8
00007FF645518EBD	48 8D15 8C81FFFF	lea rdx,qword ptr ds:[7FF645511050]
00007FF645518EC4	48 8BC8	mov rcx,rcx

Сохраняем патч, запускаем, получаем флаг

```
Enter password: password
Congratulations! FECTF{Z4_W4rud0}
Для продолжения нажмите любую клавишу . . . |
```